

Deduplication training data makes large models better

- ①: Train more efficiently (because have fewer tokens)
- ②: Avoid memorization (can mitigate - copyright, privacy concerns)

Key challenge:

⇒ Deduplication is fundamentally about comparing items to other items: computationally expensive

⇒ Need linear time algorithms to scale

improved deduplication methods:

hash_function() → { object
a hash value (integer)
hash value much smaller than item
Hash collision: $h(x) = h(y)$ for $x \neq y$

exact_deduplication() { item: string
How to match: exact match
Action: remove all but one

bloom_filter()

~~lshash~~ →

~~locally sensitive hashing~~

Bloom filter = bit array

+ hash functions

set / query bits

like molecular pathways

efficient approximate data structure for testing set membership

Features of Bloom

- Filters:
- ①: memory efficient
 - ②: can update, but can't delete.
 - ③: it return 'no', definitely 'no'
 - ④: it return 'yes', most likely 'yes'
 - ⑤: can drive the false positive rate down

#1 original item

#2 compute hash (a list of items with that hash)
#3 keep one item from each group
eg: 4 data set

expensive with time/space

Design space for deduplication: word ~~(*)~~

- ①: what is an item? (sentence, paragraph, doc?)
- ②: How to match (exact match, fractional match, ~~near~~ match)
- ③: what action to take (remove all, remove one?)

Trade-off design: efficiency and collision resistance

{ cryptographic hash functions: collision resistant, slow.

MD5, Murmurhash, Cityhash: not collision resistant, fast

⇒ build_table(): build a basic Bloom Filter (1 bit array + 1 hash function)
↳ build a Bloom Filter and inserts each item using one (a bit array)

⇒ build_table_k(): Build an advanced bloom filter with k hash functions
↳ same as build_table(), but uses k different hash functions
reduce false positive → improve accuracy marking

⇒ query_table: query basic bloom filter using 1 hash
↳ check whether an item is in a single-hash Bloom Filter

⇒ query_table_k: query bloom filter with k hash functions
↳ Reduce false positives by requiring all k evidence bits to match.

Trade-off among: Compute(k), memory(m), and false positive rate(f)

example: 170/m.a.